MSc AI Lab Project Report

End-to-End Portfolio Optimization with CNN-GARCH and RIENet

Wacil Lakbir
Sneha Basker

Supervisor: Christian Bongiorno

Emails: [add-wacil-email] | [add-sneha-email]

Git repository (maintainer access): [add-maintainer-git-link]

Date: April 9, 2026

Abstract. This project studies global minimum variance (GMV) portfolio construction under noisy covariance estimation. The practical difficulty is not the optimizer itself, but the quality of the covariance matrix given to the optimizer. We therefore compare three levels of modeling: a benchmark GARCH pipeline, a hybrid CNN-GARCH model for marginal volatility forecasting, and a cleaned covariance pipeline that combines the neural volatility model with a RIENet-style rotation-invariant correlation cleaning step. The empirical study is based on cleaned WIKI adjusted-price data from January 4, 2000 to March 27, 2018, with a 90-day rolling window and a train/test split at October 4, 2012. The main result comes from the 50-stock unconstrained GMV experiment: GMV-NN-Clean achieves the lowest annualized volatility (12.75%), while GMV-NN delivers the highest Sharpe ratio (1.4190). Additional constrained and Monte Carlo experiments confirm that the neural volatility model is competitive against standard GARCH baselines and often improves out-of-sample risk-adjusted performance. The report positions these results relative to classical GARCH, covariance-cleaning methods, and recent hybrid neural-volatility literature, then discusses remaining limitations such as transaction costs, turnover, and sensitivity to market regime changes.

1 Introduction

Minimum-variance portfolio construction is one of the cleanest settings in quantitative finance because the optimization target is explicit: reduce realized portfolio risk while maintaining acceptable diversification. In practice, however, the global minimum variance portfolio is only as good as the covariance matrix that enters the optimizer. Small errors in estimated volatilities or correlations are amplified when the covariance matrix is inverted, which can produce unstable weights, concentration, and disappointing out-of-sample behavior.

This estimation problem motivates our project. Rather than changing the downstream optimizer, we keep the GMV rule fixed and focus on improving the upstream covariance-estimation pipeline. The first reason to consider GARCH is financial interpretability. Daily returns exhibit volatility clustering, persistence, and heteroskedasticity, and the GARCH family was designed precisely to model these stylized facts [2, 3]. GARCH(1,1) remains a strong benchmark because it is parsimonious, stable, and easy to deploy on rolling windows. At the same time, pure GARCH imposes a rigid functional form and treats each asset through a small set of linear lag interactions.

Our response is a hybrid architecture. A convolutional neural network reads a 90-day return window and learns richer nonlinear temporal features than a plain handcrafted recursion. Instead of discarding econometric structure, we map the neural output into a stable GARCH-like parameterization, which preserves interpretability and imposes sensible constraints on volatility dynamics. This gives a CNN-GARCH model that aims to be more flexible than classical GARCH without becoming a black-box forecaster.

Marginal volatilities alone do not solve the GMV problem. Portfolio variance also depends on cross-asset dependence, and empirical rolling correlations are notoriously noisy in high dimension. This is why the project adds a RIENet-style covariance cleaning block inspired by rotation-invariant estimation [4, 5]. The idea is to regularize the eigenspectrum of the correlation matrix before building the covariance matrix used by the optimizer.

The report addresses the following research question: can a hybrid CNN-GARCH volatility model, combined with correlation cleaning, improve out-of-sample GMV behavior relative to a benchmark GARCH pipeline? Our answer is nuanced. The experiments show clear improvements in several settings, especially for volatility reduction in the 50-stock unconstrained GMV setup, but they do not prove that the problem is definitively solved in every market regime or portfolio constraint set.

The main contributions of the project are the following:

- a benchmark comparison between classical GARCH, hybrid CNN-GARCH, and CNN-GARCH plus RIENet-style cleaning in a common GMV framework;
- a stable neural parameterization of GARCH coefficients through the triplet (ω, C, ρ);
- an end-to-end experimental pipeline that connects synthetic pretraining, real-data fine-tuning, covariance construction, and portfolio evaluation;
- a numerical study covering the main unconstrained result, constrained portfolio variants, and Monte Carlo robustness tests.

2 State of the Art

2.1 From Markowitz to covariance regularization

Modern portfolio theory starts from the mean-variance framework of Markowitz [1]. In the GMV special case, the expected-return vector does not need to be forecast explicitly; the main object is the covariance matrix. This simplification is powerful, but it also exposes the central weakness of the method: sample covariance estimates can be unstable, especially when the number of assets is large relative to the available lookback window. Ledoit and Wolf [4] showed that shrinkage and conditioning are natural ways to reduce estimation noise and improve out-of-sample performance.

The Bongiorno end-to-end paper used in this project pushes this logic further by combining portfolio optimization with rotation-invariant covariance cleaning [5]. In that line of work, the relevant question is no longer only how to estimate covariance in isolation, but how to estimate it in a way that directly helps the downstream variance-minimization objective.

2.2 Why GARCH remains a strong baseline

ARCH and GARCH were introduced to model time-varying conditional variance in financial returns [2, 3]. They remain the canonical baseline for volatility forecasting because they capture volatility clustering with very few parameters. For daily equities, this is a sensible starting point: a one-step recursion is computationally manageable, interpretable, and aligned with rolling risk management.

For our project, GARCH is important for two reasons. First, it gives a benchmark that is financially grounded rather than purely machine-learning-based. Second, it suggests a useful inductive bias: even if a neural model learns latent features from return windows, the final variance forecast can still be constrained to behave like a stable GARCH recursion.

2.3 Neural and hybrid volatility forecasting

Recent literature increasingly combines econometric structure with neural sequence models instead of choosing one family against the other. Michańków, Kwiatkowski, and Morajda propose hybrid GRU-GARCH models for volatility and risk forecasting and report improved point forecasts on several assets [7]. Wang et al. develop a deep-learning-enhanced multivariate GARCH model in which recurrent neural components enrich a BEKK-style volatility structure [8]. The common message is that hybridization can improve nonlinear feature extraction while retaining some interpretability and structure from classical volatility models.

Our project belongs to this hybrid family, but with a different design choice. Instead of a recurrent architecture, we use a compact stack of dilated one-dimensional convolutions. The practical reason is that dilated convolutions can summarize multi-scale lag information efficiently, with a small parameter budget and parallelizable computation. The model then projects the learned representation back into a GARCH-like parameter space.

2.4 Positioning of the present work

The second Bongiorno paper relevant to this project focuses on end-to-end variance minimization and leverage resilience under a compact neural architecture [6]. Our work is closely aligned with that view of portfolio optimization as a full pipeline problem: forecasting, covariance cleaning, and portfolio construction should be evaluated together. The present project therefore sits at the

intersection of three strands: classical heteroskedastic modeling, neural volatility forecasting, and covariance cleaning for portfolio optimization.

3 Problem Formulation and Data

3.1 Data source and cleaning

The empirical dataset is built from WIKI adjusted prices. The cleaning and filtering logic is implemented in `data_stocks_WIKI_price.py`. The retained period runs from January 4, 2000 to March 27, 2018, for a final cleaned universe of 1338 stocks over 4585 trading days.

The main data filters are the following:

- rows with zero trading volume are removed before return construction;
- returns are computed from adjusted close prices;
- only assets with sufficiently complete usable histories are kept;
- assets priced below 3 USD on the first test date are excluded;
- assets with more than 20 consecutive zero returns are removed in order to reduce stale-price effects.

The temporal split used throughout the project is:

- training period: January 4, 2000 to October 3, 2012;
- test period: October 4, 2012 to March 27, 2018.

All rolling experiments use a lookback window of $W = 90$ trading days.

3.2 Notation and portfolio objective

Let $r_{i,t}$ denote the daily return of asset i at day t, and let

$$x_{i,t} = (r_{i,t-W+1}, \ldots, r_{i,t}) \in \mathbb{R}^W$$

be the 90-day input window used to forecast one-step-ahead conditional variance. Let $\hat{\sigma}^2_{i,t+1}$ denote the forecast variance for asset i, and define the diagonal volatility matrix

$$D_t = \mathrm{diag}(\hat{\sigma}_{1,t}, \ldots, \hat{\sigma}_{N,t}).$$

Let C_t denote the rolling correlation matrix, or its cleaned counterpart when a covariance cleaner is applied. The portfolio covariance estimate is then

$$\Sigma_t = D_t C_t D_t.$$

For the unconstrained GMV problem, the portfolio weights solve

$$w_t^\star = \frac{\Sigma_t^{-1}\mathbf{1}}{\mathbf{1}^\top \Sigma_t^{-1}\mathbf{1}}.$$

This formula highlights why covariance quality is crucial: any instability in Σ_t is directly transferred to the weights through matrix inversion.

In some experiments, weight constraints are added, such as per-asset caps or practical long-only restrictions. These variants do not change the conceptual point of the project. They only modify the optimizer; the central challenge remains the same, namely producing a covariance estimate that is sufficiently stable and predictive for out-of-sample risk control.

4 Methodology

4.1 Synthetic pretraining

The project first pretrains the volatility network on synthetic GARCH(1,1) series before adapting it to real equity data. The exploratory setup recovered from `Main.ipynb` uses 2000 simulated series, 2000 samples, a 90-day input window, and a 60/20/20 split between training, validation, and test subsets. This stage is not meant to replace real-data learning. Its role is to initialize the model on generic heteroskedastic dynamics so that fine-tuning on equities starts from a financially meaningful prior rather than from random weights.

The synthetic diagnostics are useful because they check whether the network learns a sensible mapping between return windows and GARCH-style parameters. The key interpretation remains the same: synthetic pretraining is mainly an initialization stage that teaches the network the geometry of heteroskedastic dynamics before real-data adaptation.

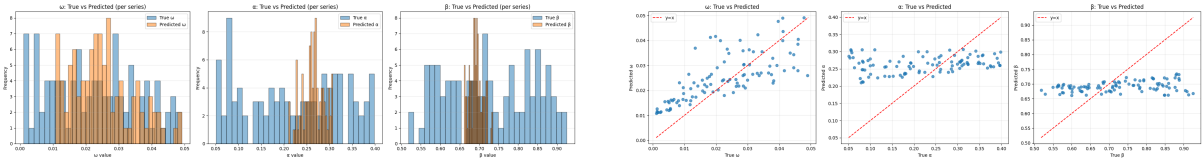

Figure 1: Synthetic pretraining diagnostics for the hybrid NN-GARCH model. The network recovers the geometry of GARCH-style heteroskedastic dynamics from simulated series, providing a financially meaningful initialization before real-data fine-tuning.

4.2 Real-data fine-tuning

Real-data training is handled in `test_on_real_time_series.ipynb`. The final fine-tuning configuration uses a 90-day window, about 20 epochs, a learning rate around 10^{-4}, weight decay around 10^{-6}, validation-based checkpoint selection, and zero-aware filtering. The preprocessing also imposes a cap on the fraction of zero returns and a small floor on the variance proxy in order to avoid degenerate log-loss behavior on illiquid or nearly constant series.

The training loss is an MSE on log-variance targets:

$$\mathcal{L}_{\text{vol}} = \frac{1}{M} \sum_{m=1}^{M} \left[\log\left(\hat{\sigma}_m^2 + \varepsilon\right) - \log\left(y_m + \varepsilon\right) \right]^2,$$

where y_m is a next-day realized variance proxy derived from squared returns and ε is a small numerical floor. The training run shows a fast and stable reduction of both training and validation loss on the WIKI panel: in the first real-data run, validation log-MSE drops from about 97 to around 7 in twenty epochs, while the held-out test score is 7.1037. The later zero-aware fine-tuning stage is even stronger, reaching a best validation loss of 4.408 and a test log-MSE of 4.415. This confirms that filtering very small r^2 targets and windows with too many zero returns improves calibration on real equities.

4.3 Hybrid CNN-GARCH architecture

The core model is implemented in `model.py`. Each input window is passed through five dilated one-dimensional convolutional layers with kernel size 3 and dilation rates $(1, 2, 4, 8, 16)$. Each block is followed by GroupNorm and LeakyReLU before a dense prediction head produces the

parameters of the final variance recursion. The model used in the main portfolio notebooks contains 14,499 trainable parameters.

The choice of five layers is deliberate. With kernel size 3 and exponentially increasing dilations, the receptive field reaches 63 days, which is large enough to capture medium-term volatility persistence while keeping the architecture compact. The 90-day window is still summarized globally by the final stacked representation, so the network can combine recent shocks with longer-range volatility context.

Instead of predicting the usual GARCH coefficients directly, the network outputs a stable triplet (ω, C, ρ) and converts it into

$$\alpha = C\rho, \qquad \beta = C(1-\rho).$$

When $0 < C < 1$ and $0 < \rho < 1$, this construction ensures

$$\alpha + \beta = C < 1,$$

which enforces a stable GARCH-like recursion. The one-step variance update is

$$\hat{\sigma}^2_{i,t+1} = \omega_{i,t} + \alpha_{i,t} r^2_{i,t} + \beta_{i,t} \hat{\sigma}^2_{i,t}.$$

This is one of the main design strengths of the project: the neural network learns nonlinear temporal features, but the final output remains financially interpretable and structurally close to GARCH.

4.4 RIENet-style correlation cleaning

Even with accurate marginal volatilities, GMV portfolios remain sensitive to noise in the correlation matrix. The covariance-cleaning part of the project therefore follows the logic of rotation-invariant estimation emphasized in [4, 5]. Let S_t be a rolling sample correlation matrix with spectral decomposition

$$S_t = V_t \Lambda_t V_t^\top.$$

The cleaner regularizes the eigenvalue structure while preserving the eigenvector basis, producing

$$\widetilde{C}_t = V_t \widetilde{\Lambda}_t V_t^\top.$$

The cleaned covariance matrix is then

$$\widetilde{\Sigma}_t = D_t \widetilde{C}_t D_t.$$

The project compares three portfolio pipelines:

- **GMV-GARCH**: benchmark volatility recursion with standard correlation input;
- **GMV-NN**: hybrid CNN-GARCH volatility with standard correlation input;
- **GMV-NN-Clean**: hybrid CNN-GARCH volatility combined with cleaned correlations.

4.5 Pseudo-code

Training pipeline

1. Simulate synthetic GARCH series and pretrain the CNN-GARCH model on one-step variance prediction.

2. Build the cleaned real-equity panel and extract rolling 90-day windows.
3. Fine-tune the model on real data with zero-aware filtering and checkpoint on validation loss.
4. Save the best neural volatility model and reuse it in the portfolio notebooks.

Daily portfolio construction pipeline

1. For each rebalancing date, compute rolling returns on the selected stock universe.
2. Predict asset-wise marginal volatilities with either GARCH or CNN-GARCH.
3. Estimate the rolling correlation matrix and optionally apply RIENet-style cleaning.
4. Assemble the covariance estimate $\Sigma_t = D_t C_t D_t$.
5. Solve the GMV optimization problem and record out-of-sample returns.

5 Numerical Results

5.1 Experimental setup

The main portfolio results are produced in the `End-to-end` notebooks. The user-specified primary result is the 50-stock unconstrained experiment from `portfolio_unconstrained_rienet.ipynb`. Supporting results come from the constrained 30-stock and 100-stock experiments in `portfolio_optimization_rienet.ipynb` and `portfolio_optimization.ipynb`. A separate Monte Carlo study in `Portfolio.ipynb` repeatedly samples 30-stock universes to test robustness.

The evaluation logic is simple. For GMV portfolios, the first objective is lower realized volatility. When two methods have comparable volatility, the second criterion is the Sharpe ratio. This is consistent with the core purpose of minimum-variance allocation.

5.2 Real-data volatility forecasting diagnostics

Before looking at portfolio returns, it is useful to verify that the volatility module itself improves on standard baselines. Figure 2 compares scratch training, the zero-aware finetuned model, ARCH/GARCH, EWMA, a naive r_{t-1}^2 predictor, and a constant-variance baseline on the same filtered real-data evaluation sample. Table 1 summarizes the corresponding metrics.

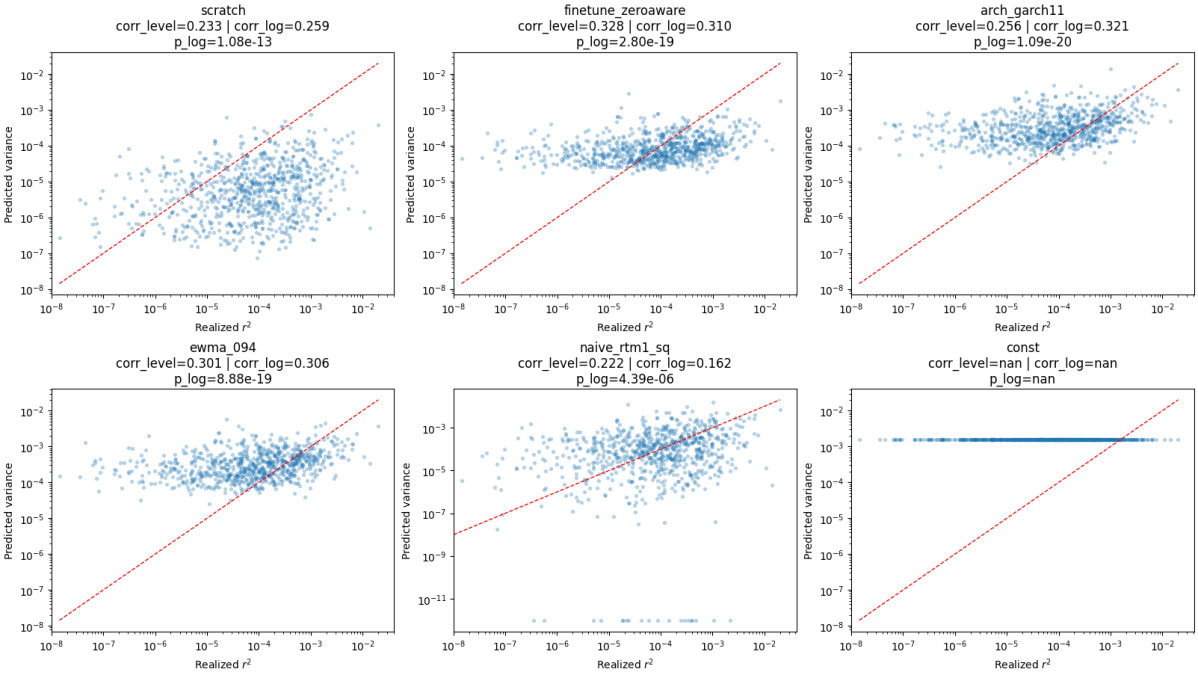

Figure 2: Predicted variance versus realized r^2 on the filtered real-data evaluation sample. The zero-aware finetuned model forms the tightest cloud among the neural methods and substantially improves over scratch training.

Table 1: Robust real-data forecasting metrics on the common filtered evaluation subset ($n = 800$).

Method	log-MSE	corr_{log}	$\text{corr}_{\text{level}}$	p-value vs ARCH
finetune_zeroaware	4.8284	0.3100	0.3280	4.86×10^{-24}
arch_garch11	7.0036	0.3215	0.2560	–
ewma_094	7.0061	0.3058	0.3005	9.71×10^{-1}
scratch	12.9517	0.2586	0.2334	3.42×10^{-15}
const	15.1234	–	–	9.31×10^{-98}
naive_rtm1_sq	15.5118	0.1615	0.2217	6.02×10^{-6}

Two points are important. First, the zero-aware finetuned model achieves the lowest log-MSE by a large margin, which means it is the best calibrated forecast in the metric that matters most for this project. Second, ARCH has a slightly higher log-correlation than the finetuned model, but its overall prediction error is substantially larger. This is why the report prioritizes log-MSE over correlation alone: a model can preserve ranking without matching the correct scale of volatility. The comparison therefore justifies the use of the finetuned neural forecaster in the portfolio experiments.

5.3 Main result: 50-stock unconstrained GMV

Table 2 contains the main result emphasized in this report.

Table 2: Main result from `portfolio_unconstrained_rienet.ipynb`.

Method	Ann. Return (%)	Ann. Vol (%)	Sharpe
GMV-GARCH	17.39	13.95	1.2468
GMV-NN	20.43	14.40	1.4190
GMV-NN-Clean	17.53	12.75	1.3755

Two conclusions matter. First, the cleaned neural pipeline produces the lowest portfolio volatility, which is the primary objective of GMV. Second, the uncleaned neural variant delivers the highest Sharpe ratio. This means the project does not yield a single uniformly dominant model across all metrics, but it does show that the neural volatility model and the cleaning stage both create measurable value.

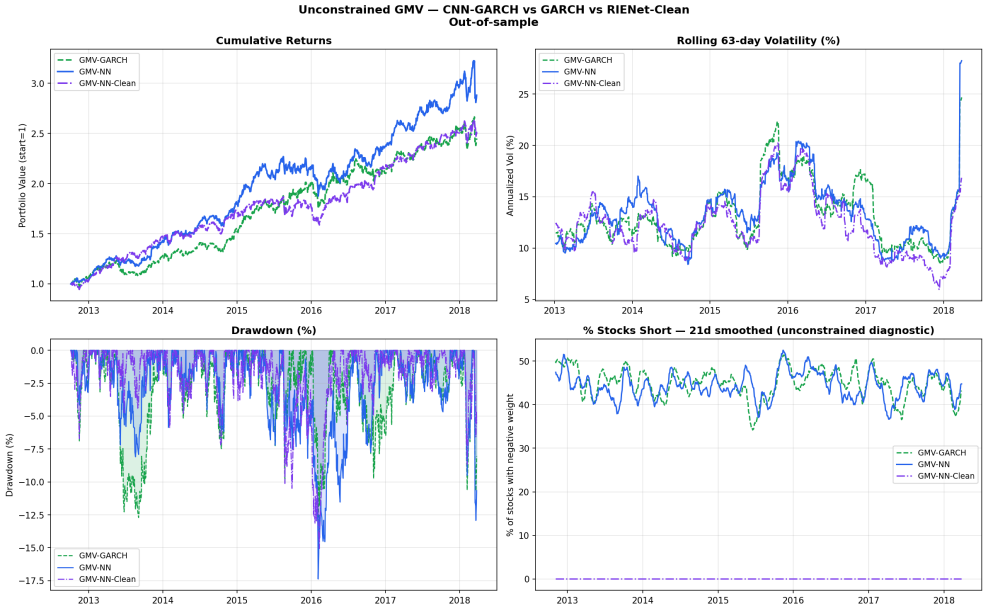

Figure 3: Cumulative performance comparison for the 50-stock unconstrained GMV experiment.

5.4 Supporting results: constrained GMV experiments

The constrained 30-stock experiment also shows a strong neural baseline.

Table 3: Constrained 30-stock result from `portfolio_optimization_rienet.ipynb`.

Method	Ann. Return (%)	Ann. Vol (%)	Sharpe
GMV-GARCH	16.73	11.45	1.4613
GMV-NN	19.19	11.44	1.6781
GMV-NN-Clean	19.22	13.02	1.4756

Here the plain neural volatility model dominates on Sharpe while matching or slightly improving volatility relative to the benchmark. The cleaned variant does not dominate in this constrained setting, which is important for an honest interpretation: covariance cleaning helps in some configurations more clearly than in others.

Table 4: Summary of the 100-stock constrained comparison from `portfolio_optimization.ipynb`.

Method	Return (%)	Vol (%)	Sharpe	Calmar
Equal-Weight	17.26	14.33	1.2044	1.0647
Inv-Var NN	17.53	12.15	1.4422	1.3357
GMV-GARCH	16.05	10.92	1.4697	1.3698
GMV-NN	18.39	10.86	1.6939	1.7504
GMV-GARCH-Clean	16.27	12.12	1.3421	1.2959
GMV-NN-Clean	17.48	12.22	1.4305	1.3247

This larger experiment reinforces the main message: the neural volatility block is competitive and can materially improve the portfolio pipeline. At the same time, the effect of cleaning is not universally monotone across all constrained settings, so the report should avoid claiming that the problem is completely solved.

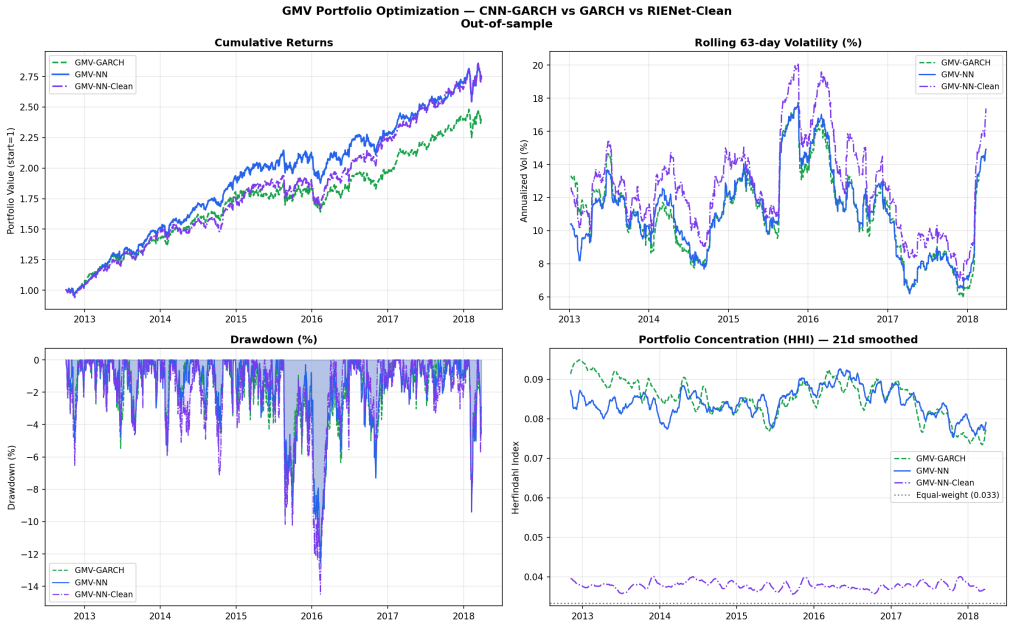

Figure 4: Example cumulative comparison from the constrained GMV experiments.

5.5 Monte Carlo robustness

The Monte Carlo analysis in `Portfolio.ipynb` repeats the evaluation over 200 random selections of 30 stocks from the 1338-stock universe. This gives a better view of how stable the conclusions are beyond a single stock subset. The reported averages are:

- mean annualized volatility: 12.7555 for NN versus 13.1485 for ARCH/GARCH baseline;
- mean Sharpe ratio: 1.1967 for NN versus 1.1525 for the baseline;
- mean paired volatility gain: +0.3930;
- mean paired Sharpe gain: +0.0442.

The paired Wilcoxon tests are strongly significant:

- volatility comparison: $W = 19563.0$, $p = 1.8875 \times 10^{-31}$;
- Sharpe comparison: $W = 14172.0$, $p = 2.4582 \times 10^{-7}$.

These results support the claim that the neural model improves the volatility side of the portfolio problem in a statistically meaningful way over repeated resampling experiments.

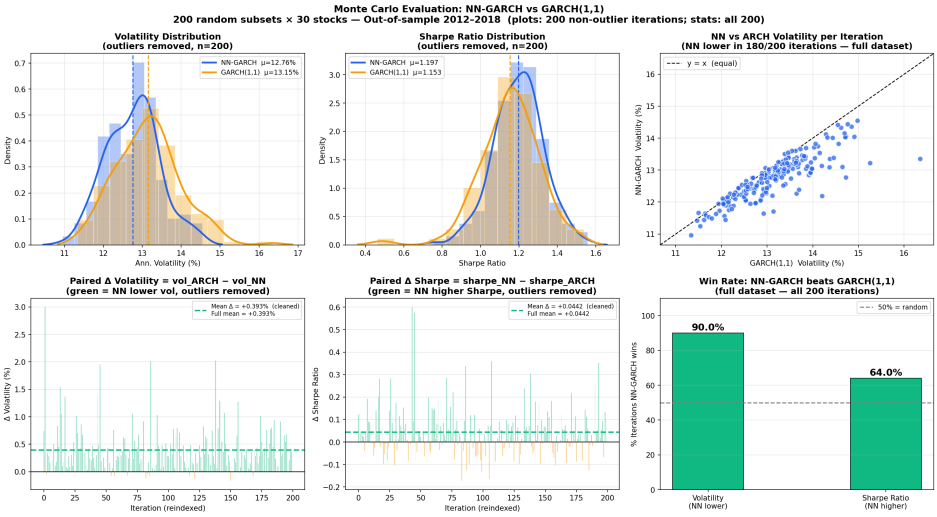

Figure 5: Monte Carlo distributions used for the paired Wilcoxon robustness study.

5.6 Computational resources and cost

The notebooks report CPU runtimes for the main experiments. In the 50-stock unconstrained notebook, volatility precomputation takes about 70.6 seconds for CNN-GARCH and 1.8 seconds for the benchmark GARCH recursion. In the constrained 30-stock notebook, the corresponding numbers are about 66.1 seconds and 0.6 seconds. In the 100-stock notebook, the CNN-GARCH pass takes about 114.9 seconds and the GARCH recursion about 4.9 seconds. The 200-run Monte Carlo study takes about 42.3 minutes on CPU.

These numbers show a clear trade-off. The neural model is slower than the benchmark, but the computational burden remains realistic for an academic daily-frequency pipeline. Monetary cost was not tracked because the experiments were run locally rather than through paid cloud infrastructure. If required for the final submission, the hardware description can be added here: `[fill local CPU / RAM / OS]`.

6 Ethical and Societal Impact

This project is mainly methodological, but it still has a positive ethical and societal dimension because better risk estimation can support more disciplined portfolio construction. The hybrid design keeps an interpretable GARCH structure at the output stage, which makes the model easier to audit and explain than a fully opaque black-box forecaster. The project also encourages reproducibility: the optimization rule is fixed, the covariance-estimation pipeline is explicit, and the code can be inspected end to end.

From an applied perspective, the present framework is a strong basis for more realistic investment settings. Long-only constraints, turnover penalties, transaction-cost models, and exposure

controls can be added on top of the same forecasting pipeline. In that sense, the project is best viewed as a transparent research contribution for portfolio-risk management, with clear room for extension toward practical and mandate-compatible implementations.

7 Project Organization

This section should be finalized with the exact division of labor agreed by the team before submission. A report is stronger when it documents not only the final model, but also how the project was organized over time.

7.1 Task distribution draft

Table 5: Draft template for task distribution. Replace the placeholders with the exact final split.

Task	Main responsibility	Notes
Literature review and state of the art	[fill]	Bongiorno papers, GARCH, covariance cleaning, hybrid NN literature
Data cleaning and preprocessing	[fill]	WIKI panel cleaning, asset filters, train/test split
Synthetic pretraining	[fill]	Simulated GARCH data generation and first model fit
Real-data fine-tuning	[fill]	Zero-aware filtering, validation checkpoints, diagnostics
Portfolio evaluation	[fill]	GMV experiments, constrained/unconstrained comparisons, robustness
Poster and report writing	[fill]	Slides, poster, tables, final narrative

7.2 Time spent draft

Table 6: Editable hours table.

Task	Wacil (h)	Sneha (h)	Total (h)
Reading and planning	[fill]	[fill]	[fill]
Implementation and debugging	[fill]	[fill]	[fill]
Training and experimentation	[fill]	[fill]	[fill]
Evaluation and figure production	[fill]	[fill]	[fill]
Writing and presentation	[fill]	[fill]	[fill]
Total	[fill]	[fill]	[fill]

7.3 Timeline draft

Table 7: Simple Gantt-style timeline for the report.

Work package	Week 1	Week 2	Week 3	Week 4	Week 5
Problem definition and reading	X	X			
Data pipeline and cleaning		X	X		
Synthetic pretraining		X	X		
Real-data fine-tuning			X	X	
Portfolio experiments				X	X
Poster and report preparation				X	X

8 Conclusion and Perspectives

The project starts from a simple observation: GMV portfolios are mostly limited by covariance estimation error rather than by the closed-form optimizer itself. This is why GARCH is an appropriate benchmark, and also why a hybrid neural extension is worth testing. The experiments show that the CNN-GARCH model improves the volatility-estimation stage in several relevant settings, and that correlation cleaning can further reduce portfolio risk in the main 50-stock unconstrained experiment.

The most careful conclusion is therefore the following. The project does not prove that the GMV estimation problem is fully solved. It does show that better marginal volatility forecasts and better-conditioned correlation estimates can materially improve out-of-sample portfolio behavior. In the main result, GMV-NN-Clean delivers the lowest volatility, while GMV-NN delivers the highest Sharpe ratio. This is exactly the kind of nuanced outcome that should be reported honestly.

The next steps are clear: add transaction costs and turnover penalties, test stricter long-only constraints, compare against shrinkage and risk-parity baselines, scale to larger universes, and explore tighter joint training between the volatility and covariance-cleaning modules. A stronger study could also include crisis-period stress tests and more detailed hardware-cost reporting.

References

[1] H. Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.

[2] R. F. Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4):987–1007, 1982.

[3] T. Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3):307–327, 1986.

[4] O. Ledoit and M. Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004.

[5] C. Bongiorno, E. Manolakis, and R. N. Mantegna. End-to-end large portfolio optimization for variance minimization with neural networks through covariance cleaning. Working paper / conference manuscript, 2025.

[6] C. Bongiorno, E. Manolakis, and R. N. Mantegna. Neural network-driven volatility drag mitigation under aggressive leverage. Working paper / ICAIF manuscript, 2025.

[7] J. Michańków, L. Kwiatkowski, and J. Morajda. Combining deep learning and GARCH models for financial volatility and risk forecasting. *arXiv preprint arXiv:2310.01063*, 2023.

[8] H. Wang, C. Liu, M.-N. Tran, and C. Wang. Deep learning enhanced multivariate GARCH. *arXiv preprint arXiv:2506.02796*, 2025.